Secure Agentic RAG Gateway ��� Build Guide

Complete Build Guide — Folder Structure, Requirements, Steps & Future Roadmap

1. Project Overview
2. What You Do NOT Need to Know
3. High-Level Architecture
4. Guardrails vs. Evaluation — Know the Difference
5. Recommended Folder Structure
6. Requirements
   - 5.1 Core Stack
   - 5.2 `requirements.txt` (starting point)
   - 5.3 Accounts / API Keys Needed
7. Step-by-Step Build Plan
   - Phase 0 — Setup (Day 1)
   - Phase 1 — Document Ingestion & Basic RAG (Days 2–4)
   - Phase 2 — Input Guardrails (Days 5–7)
   - Phase 3 — LangGraph Agent Orchestration (Days 8–11)
   - Phase 4 — Output Guardrails (Days 12–13)
   - Phase 4.5 — LLM Evaluation Layer (Days 13–14)
   - Phase 5 — Observability & Audit Logging (Days 14–15)
   - Phase 6 — Polish & Documentation (Days 16–17)
8. Minimal Viable Version (If Time is Very Short)
9. Future Integration (When You Have More Time)
   - 8.1 Guardrail Benchmark Module (Phase 9 project, folded in later)
   - 8.2 Other Future Enhancements
10. Resume Bullet Templates
11. Quick Checklist

1. Project Overview

Name: Secure Agentic RAG Gateway (“AI Firewall”)

One-line pitch: A middleware layer that sits between users and one or more LLM agents, enforcing multi-layer security (prompt-injection detection, access-controlled retrieval, output validation) on every request and response — plus an observability dashboard showing what got blocked and why.

Why it stands out: Most resume projects are “I built a RAG chatbot.” This is a *system* — it shows software architecture, security thinking, and agent orchestration together. No ML training required; everything uses pretrained models and libraries as black boxes through their APIs.

Core narrative for interviews: > “I built infrastructure that enterprises need before they can safely deploy LLM agents internally — access control, attack detection, and full audit logging, all wrapped around a LangGraph-orchestrated retrieval agent.”

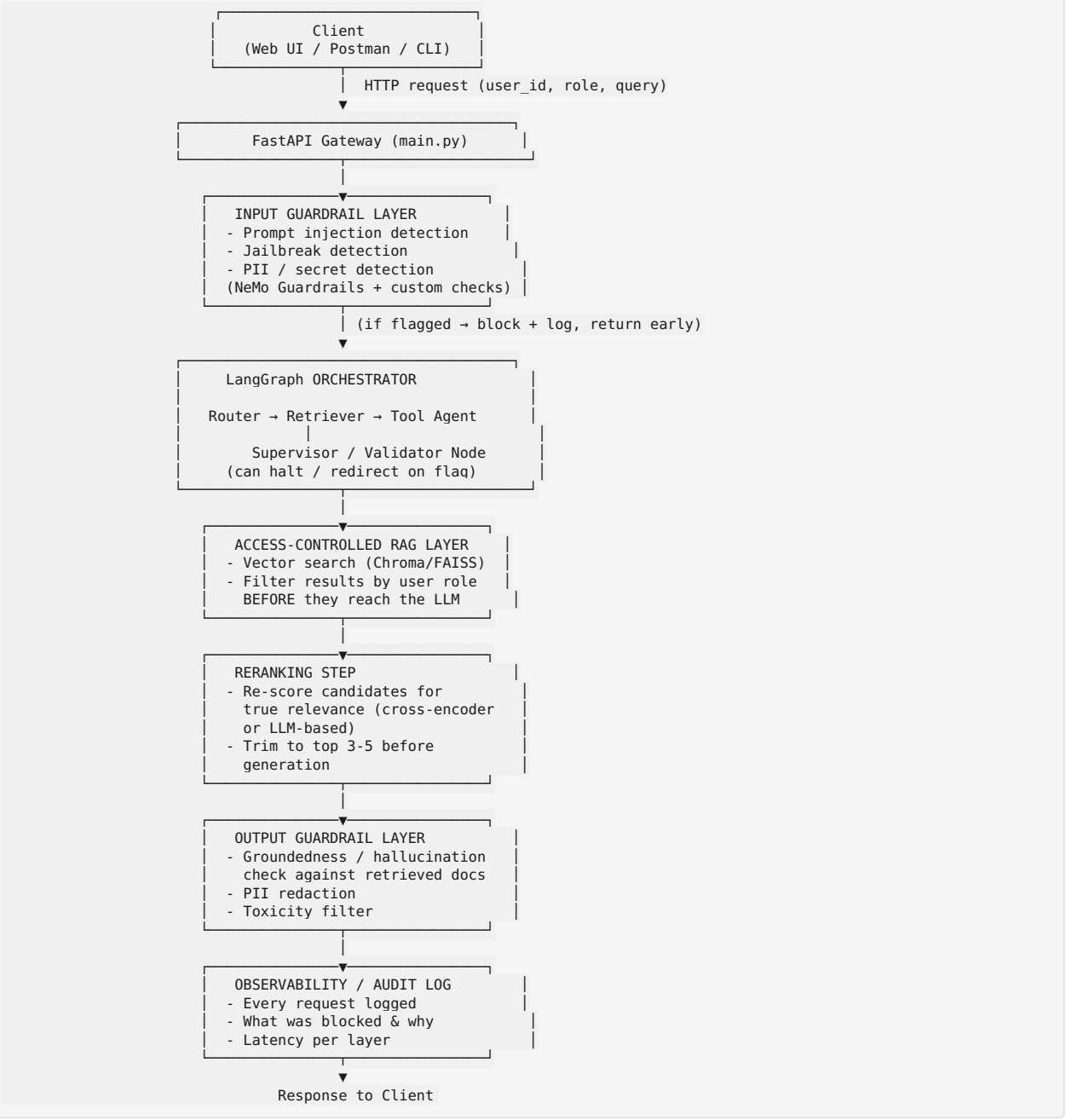
---

2. What You Do NOT Need to Know

- No transformer internals, no attention mechanism math
- No model training / fine-tuning / backpropagation
- No deep ML theory

Everything below is **API calls + software engineering**. Embeddings, classifiers, and guardrails are all used as black boxes via pre-built libraries.

3. High-Level Architecture



4. Guardrails vs. Evaluation — Know the Difference

These get confused constantly, including by candidates in interviews — so it's worth being precise, and it's also *why* this project has two separate layers instead of one.

	Guardrails	Evaluation
Question it answers	"Is this input/output *safe*?"	"Is this system *good*?"
Examples	Jailbreak detection, PII leakage, toxicity	Groundedness, retrieval accuracy, answer correctness
When it runs	On every single request, in real time, blocking	Usually on a test set, either offline or logged for review
		Hallucination, bad retrieval, low-quality

Failure mode it catches	Attacks, policy violations	answers
Tools	NeMo Guardrails, Llama Guard, Presidio	LLM-as-judge scoring, RAGAS-style metrics, golden Q&A sets

The key point: a system can be perfectly *safe* (nothing malicious got through) and still be *bad* (it retrieved the wrong document and confidently made something up). Guardrails alone won't catch that — that's what the evaluation layer is for. Building both, and being able to explain why they're different problems, is one of the strongest things you can say in an interview about this project.

---

5. Recommended Folder Structure

```
ai-firewall/
│
├── README.md
├── requirements.txt
├── .env.example
├── .gitignore
│
├── app/
│   ├── main.py                    # FastAPI entrypoint
│   ├── config.py                  # env vars, settings
│   │
│   ├── guardrails/
│   │   ├──  init  .py
│   │   ├── input checks.py        # injection/jailbreak detection
│   │   ├── output checks.py       # groundedness, PII redaction, toxicity
│   │   ├── nemo config/           # NeMo Guardrails config files (.co, .yml)
│   │   │   ├── config.yml
│   │   │   └── rails.co
│   │   └── pii_patterns.py        # regex patterns for PII
│   │
│   ├── rag/
│   │   ├──  init  .py
│   │   ├── ingest.py               # load + chunk + embed documents
│   │   ├── retriever.py            # vector search + role-based filtering
│   │   ├── reranker.py             # re-score + trim candidates before generation
│   │   ├── vectorstore.py          # Chroma/FAISS setup
│   │   └── documents/              # sample synthetic docs (HR, Finance, IT)
│   │       ├── hr/
│   │       ├── finance/
│   │       └── it/
│   │
│   ├── agents/
│   │   ├──  init  .py
│   │   ├── graph.py                # LangGraph definition (nodes + edges)
│   │   ├── router node.py
│   │   ├── retriever node.py
│   │   ├── tool agent node.py
│   │   └── validator_node.py       # supervisor / halt logic
│   │
│   ├── auth/
│   │   ├──  init  .py
│   │   └── roles.py                # user_id -> role mapping (mock or DB)
│   │
│   ├── observability/
│   │   ├──  init  .py
│   │   ├── logger.py                # structured logging
│   │   └── metrics.py               # latency, block counts, etc.
│   │
│   └── models/
│       ├──  init  .py
│       └── schemas.py               # Pydantic request/response models
│
├── dashboard/
│   └── streamlit_app.py            # observability dashboard
│
├── redteam/
│   ├── prompts.json                 # your adversarial test prompts
│   ├── run benchmark.py             # (optional/future) test guardrails
│   └── results/                     # benchmark output tables
│
├── evaluation/
│   ├── golden qa set.json           # test questions + ideal answers
│   ├── retrieval eval.py            # precision/recall on retrieved chunks
│   ├── groundedness eval.py         # LLM-as-judge faithfulness scoring
│   ├── answer quality_eval.py       # LLM-as-judge correctness scoring
│   └── results/
│       └── eval_report.csv          # scores over time, for regression tracking
│
├── tests/
│   ├── test guardrails.py
│   ├── test rag.py
│   └── test_agents.py
```

```
│
└── logs/
    └── audit_log.jsonl
```

---

6. Requirements

5.1 Core Stack

Component	Tool/Library	Purpose
API framework	FastAPI + Uvicorn	Gateway server
Orchestration	LangGraph	Multi-node agent flow
LLM access	LangChain + OpenAI/Anthropic API	Agent + generation calls
Guardrails	NeMo Guardrails	Input/output rail enforcement
Vector store	ChromaDB (or FAISS)	Document embeddings + retrieval
Embeddings	OpenAI `text-embedding-3-small` or HuggingFace `sentence-transformers`	Turn text into vectors (black-box call)
Reranking	`sentence-transformers` `CrossEncoder` (e.g. `ms-marco-MiniLM-L-6-v2`) or an LLM-based scoring prompt	Re-rank retrieved candidates by true relevance before generation
PII detection	`presidio-analyzer` (Microsoft) or regex	Detect emails, SSNs, phone numbers, etc.
Evaluation	LLM-as-judge prompts (no extra library needed) or `ragas`	Groundedness, retrieval, and answer-quality scoring
Dashboard	Streamlit	Observability UI
Logging	Python `logging` + JSON lines	Audit trail
Testing	Pytest	Unit tests for guardrails/RAG
Env management	`python-dotenv`	API keys, config

5.2 `requirements.txt` (starting point)

```
fastapi==0.115.0
uvicorn==0.30.6
langchain==0.3.0
langgraph==0.2.28
langchain-openai==0.2.0
langchain-community==0.3.0
chromadb==0.5.5
sentence-transformers==3.1.1  # used for cross-encoder reranking
nemoguardrails==0.10.1
presidio-analyzer==2.2.354
presidio-anonymizer==2.2.354
python-dotenv==1.0.1
pydantic==2.9.2
streamlit==1.38.0
pytest==8.3.3
ragas==0.1.20        # optional — pre-built RAG evaluation metrics; you can also write LLM-as-judge prompts yourself with no extra library
```

5.3 Accounts / API Keys Needed

- OpenAI or Anthropic API key (for the LLM + embeddings)
- (Optional later) HuggingFace account if you swap to open-source embedding models

---

7. Step-by-Step Build Plan

Phase 0 — Setup (Day 1)

1. Create the folder structure above.
2. Set up virtual environment, install `requirements.txt`.
3. Create `.env` with your API key.
4. Confirm a basic FastAPI app runs (`/health` endpoint returning `{"status": "ok"}`).

Phase 1 — Document Ingestion & Basic RAG (Days 2–4)

1. Create 3 small synthetic document sets: `hr/`, `finance/`, `it/` (5–10 short `.txt` or `.md` files each — you can generate these yourself or ask an LLM to draft sample HR policies, finance memos, IT runbooks).
2. Write `ingest.py`: load docs → chunk (e.g. 500 tokens) → embed → store in Chroma, **tagging each chunk with metadata:** `{"role": "hr" | "finance" | "it"}`.
3. Write `retriever.py`: given a query + user role, retrieve only chunks whose metadata role matches (or is public). This is the **access-controlled RAG** — the differentiating feature. It's just a metadata filter, no ML needed.
4. Test: same query, different roles → different (or no) results.
5. **Add a reranking step** (`rag/reranker.py`): retrieval typically pulls back more candidates than you need (e.g. top 20 by embedding similarity), and embedding similarity alone is a rough signal — it often ranks a loosely-related chunk above a more precisely relevant one. A reranker re-scores those candidates with a model built specifically for relevance ranking, and you keep only the top 3–5 to actually send to the LLM.
   - Simplest option, no ML needed: a **cross-encoder reranker** via a pre-built library call, e.g. `sentence-transformers` `CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")` — you call `.predict([(query, chunk) for chunk in candidates])` and sort by score. Still just a function call, no training.
   - Alternative: an **LLM-based reranker** — ask the LLM itself to score each candidate chunk's relevance 1–10 given the query, then sort. Zero extra libraries, pure prompting.
   - Where it sits in the flow: `retriever.py` fetches ~20 candidates (post role-filtering) → `reranker.py` re-scores and trims to the best 3–5 → those go to the tool agent as context.
   - Why this matters for interviews: it's a genuinely production-relevant detail most tutorial RAG projects skip entirely, and it directly improves your groundedness scores in the evaluation layer (Phase 4.5) — better context in means fewer hallucinated answers out.
6. Test: compare answers with and without reranking on a few ambiguous queries, and note the difference — this becomes a nice before/after data point for your README.

Phase 2 — Input Guardrails (Days 5–7)

1. Set up NeMo Guardrails with a basic `config.yml` + `rails.co` defining:
   - A jailbreak-detection rail
   - A topic-restriction rail (refuse off-topic requests)
2. Write `input_checks.py`:
   - Call NeMo Guardrails on incoming query
   - Add a simple regex/PII check (emails, phone numbers, credit-card-like patterns) using `presidio-analyzer`
   - Add a **prompt-based classifier** as a backup: ask the LLM directly, *"Classify this input as SAFE or UNSAFE for a corporate assistant: {input}. Respond with one word."* — no training required.
3. If any check flags → return an early "blocked" response, log it, stop processing.

Phase 3 — LangGraph Agent Orchestration (Days 8–11)

1. Define graph nodes: `router_node` → `retriever_node` → `tool_agent_node` → `validator_node`.
2. `router_node`: classify which document domain the query belongs to (hr/finance/it) — a simple LLM call. The classifier prompt should define each role in plain terms so the LLM has a clear boundary to classify against, for example:

```
ROLE DEFINITIONS = (
    "- 'hr': Human resources, employee benefits, leave policies, payroll.\n"
    "- 'finance': Budgets, revenue reports, accounting, and financial compliance.\n"
    "- 'it': Technical support, runbooks, VPN, software access, and hardware issues.\n\n"
)

CLASSIFIER PROMPT = (
    "Classify the following employee query into exactly one department "
    "based on these definitions:\n\n" + ROLE_DEFINITIONS +
    "Query: {query}\n"
    "Respond with only one word: hr, finance, or it."
)
```

This same `ROLE_DEFINITIONS` block should be reused in three places, so the boundaries stay consistent across the system: the router's classification prompt (above), the document metadata tagging step during ingestion (Phase 1), and the `auth/roles.py` mapping of `user_id` → role. Keeping one shared definition prevents the router classifying a query as `finance` while the retriever's metadata tags use a slightly different boundary for what counts as `finance`. 3. `retriever_node`: calls your Phase 1 retriever with the user's role. 4. `tool_agent_node`: LLM generates an answer using retrieved context (standard RAG generation call). 5. `validator_node` (the "supervisor"): checks the guardrail flags so far; can halt/redirect the flow if something looks wrong (e.g., no relevant docs found for this role → refuse rather than hallucinate). 6. Wire it all together with LangGraph's `StateGraph`.

Phase 4 — Output Guardrails (Days 12–13)

1. `output_checks.py`:
   - **Groundedness check**: ask the LLM, *"Does this answer follow only from this context? {context} / {answer}. Yes/No"* — again just an LLM call, no ML math.
   - **PII redaction**: run `presidio-anonymizer` on the response before returning it.
   - **Toxicity filter**: simple keyword list or another prompt-based check.
2. If ungrounded or unsafe → don't return the raw answer; return a fallback message and log the event.

Phase 4.5 — LLM Evaluation Layer (Days 13–14)

This is a distinct concern from guardrails and worth calling out explicitly. **Guardrails protect the system from attacks. Evaluation tells you whether the system is actually good** — is it retrieving the right documents, is it grounded in evidence, is it correct. A system can be perfectly safe and still be useless if it's hallucinating or retrieving garbage. Building both, and being able to explain the difference, is a strong interview signal.

1. **Retrieval evaluation** (`evaluation/retrieval_eval.py`):
   - Build a small set of test queries where you already know which document(s) *should* be retrieved.
   - For each query, check whether the correct chunk appears in the top-k retrieved results.
   - Report retrieval precision/recall. This catches a different failure mode than groundedness — bad retrieval vs. bad generation.
2. **Groundedness / faithfulness evaluation** (`evaluation/groundedness_eval.py`) — upgrade of the Phase 4 output check:
   - Instead of a yes/no prompt, use an **LLM-as-judge** pattern: ask the LLM to score the answer 1–5 on how well it's supported by the retrieved context, with a short justification.
   - Log this score for every request (this is the same RAGAS-style "faithfulness" metric used in production RAG systems).
3. **Answer quality evaluation** (`evaluation/answer_quality_eval.py`):
   - Build a small "golden set" — 20–30 questions with ideal answers you write yourself (`evaluation/golden_qa_set.json`).
   - Use an LLM-as-judge prompt to score your system's actual answers against the ideal answers on correctness/completeness (this mirrors RAGAS's "answer relevancy" metric).
4. **Regression tracking** (`evaluation/results/eval_report.csv`):
   - Every time you change a prompt, swap a model, or tweak the retriever, re-run the full eval set and compare scores to the previous run.
   - This is a genuinely senior-engineer habit — most people never re-test after changes. Even a simple CSV of scores-over-time is enough to demonstrate it.

Folder addition:

```
evaluation/
├── golden qa set.json         # test questions + ideal answers
├── retrieval eval.py          # precision/recall on retrieved chunks
├── groundedness eval.py       # LLM-as-judge faithfulness scoring
├── answer quality_eval.py     # LLM-as-judge correctness scoring
└── results/
    └── eval_report.csv        # scores over time, for regression tracking
```

Note on ML knowledge: none of this requires training anything. "LLM-as-judge" just means you send the LLM a prompt asking it to score another LLM's output — it's still just an API call. Precision/recall here is simple counting (how many of the top-k retrieved chunks were actually relevant), not statistics theory.

Phase 5 — Observability & Audit Logging (Days 14–15)

1. `logger.py`: write every request as a JSON line to `logs/audit_log.jsonl` — include: timestamp, user, role, query, blocked

(y/n), which layer blocked it, latency per stage, final response (redacted if needed).
2. `metrics.py`: aggregate the log into counts — total requests, blocked count, block reasons breakdown, average latency added by guardrails.
3. Build a Streamlit dashboard (`dashboard/streamlit_app.py`) reading the log file and showing:
   - A table/log of recent requests
   - A bar chart of block reasons
   - Latency breakdown per layer

Phase 6 — Polish & Documentation (Days 16–17)

1. Write a clear `README.md`: architecture diagram, setup instructions, example requests (`curl`/Postman), screenshots of the dashboard.
2. Add a short **"Results"** section: even without a full benchmark, run 15–20 adversarial prompts you write yourself against the finished system and report simple stats, e.g. *"blocked 18/20 attack attempts, 1 false positive on legitimate finance query."*
3. Push to GitHub with a clean commit history (don't dump everything in one commit — shows real engineering process).

---

8. Minimal Viable Version (If Time is Very Short)

If you're pressed for time, cut in this order and still ship something strong:

1. **Keep:** Phases 0, 1, 2, 3 (setup, RAG + access control, input guardrails, LangGraph orchestration) — this alone is a solid project.
2. **Simplify:** Output guardrails → just PII redaction + one groundedness prompt check (skip toxicity filter).
3. **Cut first if needed:** Streamlit dashboard → replace with a simple printed summary table from the log file.
4. **Never cut:** the audit log (`logs/audit_log.jsonl`) — it's cheap to build and is your proof of "this actually works," even without a dashboard.
5. **Cheapest high-value add if you have any slack left:** run 15–20 self-written adversarial prompts against your finished system and put the pass/fail numbers in the README. This single paragraph does more for your resume than most entire optional features.
6. **Second-cheapest high-value add:** the reranking step. It's a handful of lines (retrieve top-20 → `CrossEncoder.predict()` → sort → keep top 5) and is one of the few genuinely production-relevant details most tutorial RAG projects skip — worth keeping even under time pressure.

---

9. Future Integration (When You Have More Time)

These are **not required now** — build the core Firewall first, note these as "planned enhancements" in your README, and add them later if time allows.

8.1 Guardrail Benchmark Module (Phase 9 project, folded in later)

- Build `redteam/run_benchmark.py`: run your adversarial prompt set against multiple guardrail strategies:
  - Raw LLM (no protection) — baseline
  - NeMo Guardrails
  - Llama Guard (via API or local model call)
  - Your own prompt-based classifier
- Compute precision, recall, false-positive rate, and average latency for each.
- Output a results table (`redteam/results/comparison.csv`) and a short write-up.
- Update the Firewall's input-layer choice based on whichever approach wins — and note in your README: *"Input guardrail strategy selected based on a controlled benchmark across 80 adversarial prompts (see /redteam)."*

8.2 Other Future Enhancements

- **Multi-agent expansion**: split the single tool agent into specialized agents (HR agent, Finance agent, IT agent) coordinated by the router, closer to a true multi-agent system.

- **Rate limiting / abuse detection**: flag users sending repeated attack attempts, add temporary lockout logic.
- **Confused-deputy protection**: validate that a tool call's parameters match the agent's stated intent before executing it.
- **Swap to open-source models**: replace OpenAI calls with a locally hosted model (e.g. via Ollama) to demonstrate provider-agnostic design.
- **CI pipeline**: GitHub Actions running `pytest` on every push, including a guardrail regression test (make sure new changes don't reduce block rate on your redteam set).
- **Deploy**: containerize with Docker, deploy to a small cloud instance (Render/Railway/Fly.io) so you have a live demo link for your resume.

---

10. Resume Bullet Templates

If you skip the benchmark: > Built a secure agentic RAG gateway using NeMo Guardrails for injection/jailbreak detection, role-based access-controlled document retrieval, and LangGraph-orchestrated multi-step agents with a validator node; logged full request/response audit trail and blocked 18/20 self-authored adversarial attack attempts with 1 false positive.

If you complete the benchmark integration: > Built a secure agentic RAG gateway (LangGraph + NeMo Guardrails) with role-based document access and a custom guardrails layer; benchmarked 3 guardrail strategies against 80 adversarial prompts, selected the approach with the best precision/recall/latency tradeoff, and deployed it as production-style middleware with full audit logging and an observability dashboard.

If you also add the evaluation layer: > Built a secure agentic RAG gateway (LangGraph + NeMo Guardrails) combining a security layer (input/output guardrails, role-based access control) with a separate LLM evaluation layer (retrieval precision/recall, LLM-as-judge groundedness scoring, and answer-quality testing against a golden Q&A set) to track both system safety and system quality independently.

---

11. Quick Checklist

- [ ] Folder structure created
- [ ] `.env` + API keys working
- [ ] Synthetic documents created (hr/finance/it)
- [ ] Ingestion + role-tagged vector store working
- [ ] Access-controlled retrieval tested (different roles → different results)
- [ ] Reranking step added and tested (before/after comparison noted)
- [ ] Input guardrails (NeMo + PII + classifier) working
- [ ] LangGraph orchestration wired end-to-end
- [ ] Output guardrails (groundedness + PII redaction) working
- [ ] Evaluation layer: retrieval precision/recall test set built
- [ ] Evaluation layer: LLM-as-judge groundedness scoring implemented
- [ ] Evaluation layer: golden Q&A set + answer-quality scoring
- [ ] Audit logging to `audit_log.jsonl`
- [ ] Dashboard (or fallback summary table)
- [ ] README with architecture diagram + setup + results section
- [ ] Pushed to GitHub with clean commit history
- [ ] (Future) Benchmark module folded in
- [ ] (Future) Deployed live demo link